KAMMAVARI SANGHAM(R) - 1952

K. S. INSTITUTE OF TECHNOLOGY

An Autonomous Institution under VTU, Belagavi & Approved by AICTE, New Delhi.
Accredited by NBA (CSE & ECE) & NAAC with **A⁺ Grade**
#14, Raghuvanahalli, Kanakapura Road, Bengaluru - 560 109

Department of Computer Science and Engineering
Machine Learning
Practical Assignment(BCS602)

Learning and Discovering Patterns in Handwritten Digits using KNN and FP-Growth

Batch No.: 2025_CSE_16

Arun Chavan	**1KS23CS019**
Bhuvan B	**1KS23CS025**
Charan Sai Tej K V	**1KS23CS029**
Charan Teja G S	**1KS23CS030**

Guided By:
Sheba Jebakani
Assistant Professor
CSE Dept

Contents

- Problem Statements
- Objective
- Description of Problem statements
- Approach
- K-Nearest Neighbors
- FP-Growth
- Dataset
- Work Flow
- Program Execution

Problem Statement

Chapter 02-Q4:

Use the Digits data set available under sklearn:
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
Consider 10% of the data for training (179 samples). Each pattern is an 8×8–sized character, where each value is an integer in the range 0 to 16. Convert it into binary form by replacing a value below 8 by 0 and other values (≥ 8) by 1.

a. Use these 179 patterns with labels and the remaining without labels for this subtask. Use KNN and label the patterns without labels. Obtain the percentage classification accuracy. Perform this task with values from the set {1, 3, 5, 10, 20}.

b. Obtain the frequent itemsets for these 179 patterns using FP-growth by viewing each binary pattern as a transaction of 64 items. Repeat this task with different minsup values from {0.1, 0.3, 0.5, 0.7}.

Objective

The main objectives of this experiment are:

- To convert grayscale digit images into binary format for simpler analysis
- To perform classification using a small labeled dataset (10%)
- To apply K-Nearest Neighbors with different values of k: k = 1, 3, 5, 10, 20
- To evaluate classification accuracy on the remaining data
- To apply FP-Growth for pattern mining
- To find frequent pixel combinations using different minimum support values: Minsup = 0.1, 0.3, 0.5, 0.7.
- To analyze how parameter changes (k and minsup) affect results
- To understand how learning and pattern discovery behave on the same dataset

Description Of Problem statement

This experiment uses handwritten digit images.

- Each image is 8×8 pixels
- Pixel values range from 0 to 16
- Data is converted into binary:
  - value $< 8 \rightarrow 0$
  - value $\geq 8 \rightarrow 1$

Two main tasks are performed:

- Classification → predict digit labels
- Pattern mining → find frequently occurring pixel groups

Approach

1. Classification (Using KNN)

What:

- Use K-Nearest Neighbors to classify digit images

Why:

- To predict labels when only a small amount of data is labeled
- To study how accuracy changes with different k values

How:

- Convert images into binary feature vectors
- Use 10% data for training and the rest for testing
- Predict labels based on nearest neighbors

Approach

2. Pattern Mining (Using FP-Growth)

What:

- Use FP-Growth to find frequent pixel patterns

Why:

- To identify commonly occurring pixel combinations in digit images
- To understand structure in the data without using labels

How:

- Convert each image into a set of active pixels
- Apply FP-Growth with different support values
- Extract frequent itemsets

K-Nearest Neighbors (KNN)

Core Idea

- K-Nearest Neighbors is an instance-based learning algorithm
- It makes decisions based on stored training data

Working Principle

- Compute distance to all training samples
- Select the k closest samples
- Assign the label using majority voting

Distance Metric

- Most commonly used: Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

FP-Growth Algorithm

Core Idea

- FP-Growthis used to find frequent itemsets efficiently
- Avoids generating all candidate sets (unlike Apriori)

Working Principle

- Scan dataset to find frequent items
- Build an FP-tree (compressed representation)
- Recursively mine the tree to extract frequent patterns

FP-Tree Structure

- Nodes represent items
- Paths represent transactions
- Shared prefixes are merged → reduces redundancy

Key Concepts

- Support

$$\text{Support} = \frac{\text{Number of transactions containing itemset}}{\text{Total transactions}}$$

- Minimum Support (minsup)
  - Threshold to decide if a pattern is frequent

DataSet

The dataset used in this experiment is:

- Scikit-learn Digits Dataset

Key Details

- Total samples: 1797
- Image size:
  - 8×8 pixels
- Number of features:
  - 64 (one per pixel)
- Pixel value range:
  - 0 to 16
- Number of classes:
  - 10 (digits 0–9)

Data Representation

- Each image is flattened into a 64-dimensional vector
- Converted into binary format:
  - value $< 8 \rightarrow 0$
  - value $\geq 8 \rightarrow 1$

Work Flow

Step 1: Data Loading

- Load the Scikit-learn Digits Dataset using sklearn

Step 2: Preprocessing

- Convert pixel values into binary:
  - value $< 8 \rightarrow 0$
  - value $\geq 8 \rightarrow 1$
- Flatten each image into a 64-dimensional vector

Step 3: Data Splitting

- Select first 10% (179 samples) as training data
- Use remaining data as testing data

Step 4: Classification using KNN

- Apply K-Nearest Neighbors
- Train using labeled training data
- Predict labels for test data using:
- k = 1, 3, 5, 10, 20

Step 5: Evaluation

- Compute classification accuracy for each k
- Compare performance across different k values

Work Flow

Step 6: Data Transformation for Mining

- Convert each binary image into a transaction
- Each active pixel (value = 1) → item

Step 7: Pattern Mining using FP-Growth

- Apply FP-Growth
- Use minsup values:
- 0.1, 0.3, 0.5, 0.7

Step 8: Analysis

- Accuracy trends (KNN)
- Number and type of frequent patterns (FP-Growth)

Program

```python
def run_knn(X_train, y_train, X_test, y_test):
    k_values = [1, 3, 5, 10, 20]
    results = []

    print("\n===== TASK 1: KNN CLASSIFICATION
=====\n")
    for k in k_values:
        model = KNeighborsClassifier(n_neighbors=k)
        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        correct = np.sum(y_pred == y_test)
        total = len(y_test)
        accuracy = correct / total

        results.append((k, accuracy))

        print(f"k = {k}")
        print(f"Correct: {correct}/{total}")
        print(f"Accuracy: {accuracy:.4f}")
        print("-" * 30)

    return results
```

```python
def run_fpgrowth(X_train):
    print("\n===== TASK 2: FP-GROWTH =====\n")

    df = pd.DataFrame(X_train.astype(bool))
    df.columns = [f'pixel_{i}' for i in range(64)]

    minsup_values = [0.1, 0.3, 0.5, 0.7]
    results = {}

    for minsup in minsup_values:
        freq_itemsets = fpgrowth(df, min_support=minsup,
use_colnames=True)
        print(f"minsup = {minsup}")
        print("Number of patterns:", len(freq_itemsets))
        print(freq_itemsets.head())
        print("-" * 40)

        results[minsup] = freq_itemsets

    return results
```

Output

```
❯ python presen.py
Loading and preprocessing data...

===== TASK 1: KNN CLASSIFICATION =====

k = 1
Correct: 1289/1618
Accuracy: 79.67%
------------------------------
k = 3
Correct: 1270/1618
Accuracy: 78.49%
------------------------------
k = 5
Correct: 1265/1618
Accuracy: 78.18%
------------------------------
k = 10
Correct: 1224/1618
Accuracy: 75.65%
------------------------------
k = 20
Correct: 1142/1618
Accuracy: 70.58%
------------------------------
```

```
===== TASK 2: FP-GROWTH =====

minsup = 0.1
Number of patterns: 467082
    support    itemsets
0  0.888268  (pixel_12)
1  0.832402   (pixel_4)
2  0.832402  (pixel_11)
3  0.804469  (pixel_60)
4  0.798883  (pixel_52)
----------------------------------------
minsup = 0.3
Number of patterns: 3441
    support    itemsets
0  0.888268  (pixel_12)
1  0.832402   (pixel_4)
2  0.832402  (pixel_11)
3  0.804469  (pixel_60)
4  0.798883  (pixel_52)
----------------------------------------
minsup = 0.5
Number of patterns: 165
    support    itemsets
0  0.888268  (pixel_12)
1  0.832402   (pixel_4)
2  0.832402  (pixel_11)
3  0.804469  (pixel_60)
4  0.798883  (pixel_52)
----------------------------------------
minsup = 0.7
Number of patterns: 15
    support    itemsets
0  0.888268  (pixel_12)
1  0.832402  (pixel_11)
2  0.832402   (pixel_4)
3  0.804469  (pixel_60)
4  0.798883  (pixel_52)
----------------------------------------

Execution completed successfully.
Saved: knn_results.png and fp_patterns_grid.png
```

References

[1] Manaranjan Pradhan and U. Dinesh Kumar. Machine Learning using Python. Wiley India, 2019.

[2] Murty, M. N., and V. S. Ananthanarayana. Machine Learning: Theory and Practice. Universities Press, 2024.

[3] scikit-learn Documentation – https://scikit-learn.org